

Warp

Warp scans a repo for a plan, builds a schedule, and dispatches workers. You drive it from a Cursor agent window. The terminal in that window is where the commands run. This guide is the session, from install through stop, including how to take the plan out, edit it, and put it back.

Where the files actually are

A local agent uses your clone on disk. A cloud agent uses a Cursor virtual machine that has its own clone. In both cases Warp writes a `.warp` directory in that working copy. It does not commit that directory, and it does not push it. Product code is the only thing that becomes a commit: a Shuttle opens a branch named `warp/` and a pull request. Status writes stay local. Hundreds of beam updates are not source history.

Add the plugin snippet `assets/gitignore-snippet.txt` to the repo `.gitignore` so a later git add cannot sweep `.warp` in. To review files, open `.warp` in the Cursor file tree and download them. On a cloud VM, download before the VM is discarded. The review zip is `.warp/warp-review.zip`, written by the `bundle` command. Set `stateDir` in config to an absolute path if you want the beam outside the repo entirely.

Session, in order

Step	In the agent window	What you should see
Install	Copy the plugin to <code>.cursor/plugins/warp</code> . Copy <code>assets/config.example.yaml</code> to <code>.warp/config.yaml</code> . Reload Cursor.	Commands <code>/warp-scan</code> and <code>/warp-start</code> appear.
Scan	<code>/warp-scan</code>	Ticket count, format, <code>runState</code> stopped. <code>.warp/beam.json</code> exists.
Start	<code>/warp-start</code>	Herald posts. Shuttles claim up to <code>maxAgents</code> .
Pause	<code>/warp-pause</code>	No new claims. In-flight finishes its current step.
Download	Ask for a bundle, or download <code>.warp/WARP_PLAN.json</code> and <code>STATUS.md</code> from the file tree.	A zip or the two files on your machine.
Edit	Edit <code>WARP_PLAN.json</code> locally. Do not edit <code>beam.json</code> .	Changed deps, locks, sizes, or gates.
Upload	Drag the edited JSON into the agent chat, or drop it in the workspace as <code>WARP_PLAN.edited.json</code> . Do not commit it.	The agent can see the path.
Import	<code>/warp-import</code> on that path.	Plan replaced. Run stopped. Status kept for ids that remain.
Restart	<code>/warp-start</code>	Dispatch uses the new graph.
Stop	<code>/warp-stop</code>	Stays stopped until the next start. Other repos are unaffected.

Export, edit, import

Pause first, so a tick cannot claim while the graph is replaced. Export writes two files. Only the JSON is imported. The markdown is the brief you can hand to another model.

```
/warp-pause
/warp-export
```

Writes `.warp/WARP_PLAN.json` and `.warp/WARP_PLAN.md`. Download both. Edit the JSON. Safe fields: `deps`, `locks`, `size`, `autoMerge`, `critical`, and `gate members and checks`. Keep an id stable if you want its pull request, tokens, and minutes kept. Rename an id and Warp treats it as new.

Upload by dragging the file into the agent chat, or saving it in the workspace. Then:

```
/warp-import path/to/WARP_PLAN.edited.json
```

Import rewrites the graph, copies status onto ids that still exist, drops removed ids, and sets `runState` to stopped. Read `.warp/STATUS.md`. Start when the counts look right. Config is not replaced by an import.

What you edit, and what you do not

File	Edit	Role
.warp/WARP_PLAN.json	Yes	The file import reads.
.warp/WARP_PLAN.md	Read	Analysis brief. Not imported.
.warp/config.yaml	Yes	Model, cap, messenger. Survives import.
.warp/beam.json	No	Live status. Scripts write it.
.warp/STATUS.md	No	Done, working, left. Download to review.
.warp/journal.jsonl	No	Append-only log. Leave it out of git.

Configuration

One file, .warp/config.yaml. Defaults below. Change the yaml, then restart, so the next tick re-reads it. There is no person cap.

Key	Default	Meaning
stateDir	.warp	Beam and exports. Gitignore it. Absolute path keeps it outside the repo.
model	claude-sonnet-5.5	Coding slug. Must match the Cursor picker.
maxAgents	18	Concurrent Shuttles. The only cap.
autoMergeSizes	S, M	Auto-merge after Bugbot and CI. L and XL wait.
messenger	both	slack, teams, or both.
notify	verbose	Every claim and tick. quiet keeps alarms and stops.
runner	cloud	cloud is a Cursor VM. local is this machine.
jiraProject	empty	Pin a Jira project key, or leave blank.
bugbotRequired	true	No merge without a Bugbot pass.
maxFixAttempts	3	Then the ticket alarms.
stuckAfterMinutes	90	No update in this window raises stuck, unless waiting on approval.
respectMergeWindows	false	True makes new claims wait for a window.
mergeWindows	08:30, 13:00, 17:00	Digest times, or claim gates if the flag above is true.
pollSeconds	300	How often a running loop reconciles pull requests.
jiraMcp	atlassian	Connected Jira server name.
bitbucketMcp	bitbucket	Connected Bitbucket server name.
slackMcp / teamsMcp	slack / teams	Connected messenger names.

Status and control

/warp-status rewrites .warp/STATUS.md and status.json. Working means claimed, coding, review, fix, or waiting on approval. Done means merged or moved to Done in Jira. Left is everything else. Pause holds new work. Stop ends the run until an explicit start. Each repo has its own .warp, so stopping one project does not stop another.